

"Home work Assignment #2 ch #4"

4.1). Ans:- The three examples where multithreading is better in performance than a single thread are:

1). Web Servers:- The each request is been handled by a separate thread.

- It has a faster responses for multiple users.

2). Matrix Multiplication:-

- we divide like the matrix into parts.
- where each thread computes parts at faster pace.

3). File Download Managers:-

- The multiple threads are downloading different parts.
- It improves speed while using the parallel network usage.

4.2). (a). Ans:- $N=2$

$$P=0.6 \quad \text{So Speedup} = \frac{1}{\frac{0.6}{2} + 0.4} = \boxed{1.43}$$

$$(b). \underline{\text{Ans:-}} \quad \text{So Speedup} = \frac{1}{\frac{0.6}{4} + 0.4} = \boxed{1.82}$$

4.3). Ans:- It does Task Parallelism as different threads do different tasks. like handle multiple/different request.

4.4). Ans:- The differences are:-

User Threads	Kernel level Threads
→ Managed in userspace.	→ Managed by OS
→ Faster to create	→ Slower to create
→ If 1 Thread gets blocked all Threads get blocked.	→ In this if 1 Thread gets blocked others are not affected. so Independent execution.

So better choice is user level ^{for when} it is faster, use simple apps and kernel level threads for multicore and real parallelism.

GENIUS

4.5). The actions that are taken by a kernel to context-switch b/w kernel level threads is that

- Ans:-
- save the registers with the current thread.
 - save the program counter (PC).
 - update the thread state.
 - then load new thread registers after this.
 - At last resume the execution.

4.6). Ans:- → Thread uses:

- stack
- PC. (Program Counter).
- Registers

∴ Threads are lightweight and processes are heavyweight.

→ The process uses:

- Address space.
- file
- heap
- global

4.7). Ans:- Yes definitely the binding is required.

∴ so in real time thread needs:

- The guarantee of CPU access.
- No delay.

∴ so that's why binding to LWP ensures the scheduling.

4.8). Ans:- The multithreading is not useful when a single CPU bounded task

in this no parallel work so no gain.

Heavy synchronization

It takes too much locking and must slow.

4.9). Ans:-

Even on a single CPU.

so it will improve responsiveness and also there is an overlap b/w I/O and the CPU.

GENIUS

4.10). component shared (status).

<u>Ans:-</u>	Heap	Yes
	Register	No
	Stack	No
	Global variable	Yes

4.11). Ans:- No it is for only the user level threads and so kernel doesn't see the threads so it cannot run on multiple cores that's why multiple user-level threads perform better.

4.12). Ans:- so in this case using the processes is better because for several reasons:

It can crash on isolation

In terms of security.

Memory protection.

As threads would share memory that is risky can lead to race conditions due to common memory.

4.13). Ans:-

Yes it is possible that we have concurrency but not parallelism because concurrency means tasks overlap and in parallelism means tasks run simultaneously so for ex:- single CPU running on multiple threads.

4.14). Ans:- (1). Ans:- $P=0.4$

$$N=8 \rightarrow = \frac{1}{\frac{0.6+0.4}{8}} = 1.54$$

$$, N=16 \rightarrow = \frac{1}{\frac{0.6+0.6}{16}} = 1.60$$

(2). Ans:- $P=0.67$

$$N=2 \rightarrow \frac{1}{\frac{0.33+0.67}{2}} = 1.50$$

$$, N=4 \rightarrow \frac{1}{\frac{0.33+0.67}{4}} = 2.11$$

$$(3). \underline{\text{Ans:-}} P=0.9 \rightarrow \frac{1}{\frac{0.1+0.9}{4}} = 3.08$$

$$, N=8 \rightarrow \frac{1}{\frac{0.1+0.9}{8}} = 4.71$$

GENIUS

4.15).

Problem

TYPE of Parallelism

Ans:-

Thumbnail2 Thread Task

Matrix Transpose Data

Network read/write Task

Fork-Join Sum Data

Grand Central Dispatch Task

4.16).

Ans:-

I/O Threads

So 1 Thread each

Input = 1

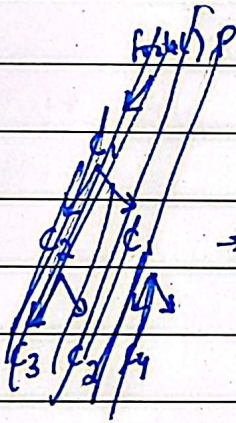
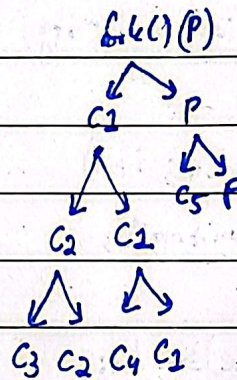
output = 1

So 4 CPU Threads

4 Threads

because 4 cores $\rightarrow$ maximum utilization.

4.17).

Ans:- $\rightarrow$ 1 Thread created here $\rightarrow$ 1 Thread created only here.

So Total Processes = 6

(P, C1, C2, C3, C4, C5)

Total Thread created = 1

4.18). Ans:- Interm of Linux vs windows :So Linux $\rightarrow$ uses `clone()`

- Not quite a strict difference.
- It is quite flexible.

So For Windows $\rightarrow$ It has separate processes + threads

- It has a better clear structure.

 $\rightarrow$ so we can infer that Linux is much more flexible and windows has a better structure.

GENIUS